

Automated Logical Inference

Lecture 15

CS 4880

Kirk Duran

Last Time: We Could Prove Things by Checking Every World

Suppose our knowledge base contains

$$A \rightarrow B, \quad A.$$

To test whether

$$KB \models B,$$

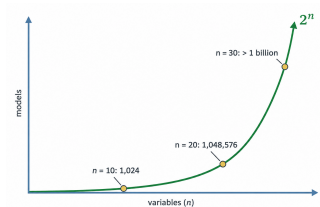
truth-table inference checks every assignment of truth values.

Correct, but expensive

With n propositional symbols, there are

$$2^n$$

possible models.



Key idea

Semantic inference asks whether a conclusion is true in every model. Today we ask whether we can derive conclusions without enumerating all models.

A Logical Agent Needs an Inference Engine

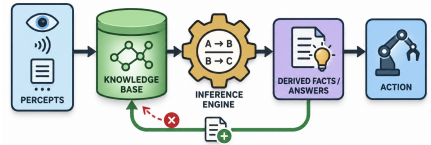
A logical agent stores sentences in a knowledge base, but storing knowledge is not enough.

The agent also needs procedures that can answer questions such as:

- ▶ Is this conclusion implied by what I know?
- ▶ Which new facts can I derive?
- ▶ Can I prove a particular goal?

Today

We turn logical inference into algorithms that manipulate sentences directly.

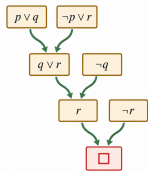


Two Ways to Reason

Model checking

Ask whether the knowledge base and query have the right truth values across possible worlds.

Think: inspect models.



Theorem proving

Apply inference rules that transform known sentences into new sentences.

Think: manipulate symbols.

Key idea

A proof system is useful when a small sequence of syntactic steps can replace an enormous semantic search.

Why Normalize Logical Sentences?

Consider two equivalent statements:

$$A \rightarrow B$$

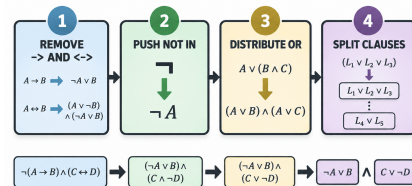
and

$$\neg A \vee B.$$

A human can recognize that these mean the same thing. An inference algorithm is easier to write if equivalent sentences are converted into a predictable structure first.

Normalization

Rewrite many different logical expressions into one standard form before inference begins.



Literals and Clauses

Definition

A **literal** is either a propositional symbol or its negation.

Literals

$A, \quad \neg B, \quad C$

Clause

A disjunction of literals:

$$A \vee \neg B \vee C$$

Key idea

Resolution operates on clauses, so clause structure is the basic unit of automated inference in this lecture.

Conjunctive Normal Form

Definition

A sentence is in **conjunctive normal form (CNF)** when it is a conjunction of clauses, where each clause is a disjunction of literals.

$$(A \vee \neg B) \wedge (C \vee D) \wedge (\neg A \vee E)$$

AND of ORs



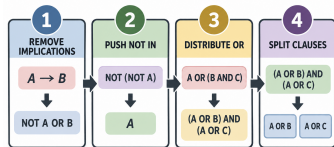
CNF Conversion: The Recipe

A common propositional conversion pipeline is:

1. Eliminate biconditionals $\leftrightarrow$.
2. Eliminate implications $\rightarrow$.
3. Move negations inward using De Morgan's laws and double negation.
4. Distribute $\vee$ over $\wedge$.

Important

Each rewrite preserves logical equivalence. We are changing the sentence's form, not its meaning.



Worked CNF Conversion: Start with an Implication

Convert

$$A \rightarrow (B \wedge C)$$

to CNF.

Step 1: eliminate implication

$$\neg A \vee (B \wedge C)$$

We are not done because the OR contains a conjunction.

Next: distribute $\vee$ over $\wedge$.

Worked CNF Conversion: Distribute

Continue from

$$\neg A \vee (B \wedge C).$$

Distribution gives

$$(\neg A \vee B) \wedge (\neg A \vee C).$$

Now the expression is a conjunction of clauses.

Clause set view

$$\{\neg A, B\}, \quad \{\neg A, C\}$$

Key idea

CNF lets the inference engine treat the knowledge base as a collection of clauses instead of arbitrary nested expressions.

Quick Check: Which One Is CNF?

Which of the following is already in conjunctive normal form?

1. $(A \wedge B) \vee C$
2. $(A \vee B) \wedge (\neg C \vee D)$
3. $A \rightarrow (B \vee C)$

Quick Check: Which One Is CNF?

Which of the following is already in conjunctive normal form?

1. $(A \wedge B) \vee C$
2. $(A \vee B) \wedge (\neg C \vee D)$
3. $A \rightarrow (B \vee C)$

Answer

$$(A \vee B) \wedge (\neg C \vee D)$$

is an AND of OR-clauses.

Resolution: One Small Rule with a Big Job

Suppose the knowledge base contains

$$A \vee B$$

and

$$\neg B \vee C.$$

If B is true, the second clause requires C . If B is false, the first clause requires A .
Either way,

$$A \vee C$$

must be true.

Key idea

Resolution removes a pair of complementary literals and combines what remains.

The Resolution Rule

Definition

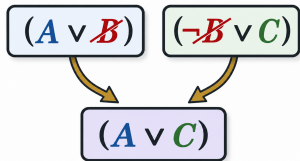
If two clauses contain complementary literals, resolution derives a new clause containing the remaining literals.

$$(L \vee A_1 \vee \cdots \vee A_m)$$

$$(\neg L \vee B_1 \vee \cdots \vee B_n)$$

resolve to

$$A_1 \vee \cdots \vee A_m \vee B_1 \vee \cdots \vee B_n.$$



Worked Resolution Step

Start with

$$\neg Rain \vee Wet$$

and

$$Rain.$$

Treat the second statement as the one-literal clause ($Rain$).

Resolve on the complementary literals $Rain$ and $\neg Rain$.

Removing that complementary pair leaves

$$Wet.$$

This should look familiar

Resolution has reproduced modus ponens using a general clause-level rule.

But How Do We Prove a Query?

Suppose we want to determine whether

$$KB \models Q.$$

Resolution is usually used by **refutation**:

1. Add the negation of the query, $\neg Q$, to the knowledge base.
2. Repeatedly apply resolution.
3. If this produces a contradiction, then Q must follow from the original knowledge base.

Assume the conclusion is false and see whether the knowledge base becomes impossible.

Resolution by Refutation: Running Example

Knowledge base:

$$A \rightarrow B$$

$$B \rightarrow C$$

$$A$$

Query:

$$C?$$

Convert the knowledge base to clauses:

$$\neg A \vee B$$

$$\neg B \vee C$$

$$A$$

Then add the negated query:

$$\neg C.$$

Refutation Setup: Clause Form

The proof search begins with four clauses:

$$\neg A \vee B$$

$$\neg B \vee C$$

$$A$$

$$\neg C \quad (\text{assumed for contradiction})$$

Key idea

From this point forward, every inference step uses the same resolution rule.

Refutation Trace: Resolve the Query Backward

Start with

$$\neg B \vee C$$

and the assumed negated query

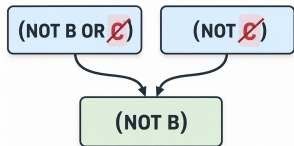
$$\neg C.$$

Resolving on C gives

$$\neg B.$$

Interpretation

If C were false, then the rule $B \rightarrow C$ says B would also have to be false.



Refutation Trace: Continue the Chain

Now resolve

$$\neg A \vee B$$

with

$$\neg B.$$

Resolving on B gives

$$\neg A.$$

But the knowledge base also contains

$$A.$$

The assumption $\neg C$ is forcing us toward a direct contradiction.

The Empty Clause Means Contradiction

Resolve

A

with

$\neg A.$

Nothing remains:

$\square$

The **empty clause** represents an impossible clause: a disjunction with no literal that could make it true.

Conclusion

$KB \wedge \neg C$ is unsatisfiable, therefore

$KB \models C.$

Why Refutation Works

To prove

$$KB \models Q,$$

we ask whether there exists any model in which

$$KB \wedge \neg Q$$

is true.

If no such model exists, then every model satisfying KB must satisfy Q .

Key idea

Resolution is syntactic, but refutation connects it directly back to the semantic definition of entailment from the previous lecture.

Resolution Is Sound and Complete for Propositional Logic

Sound

Every clause derived by resolution is logically implied by its parent clauses.
It does not invent false consequences.

Complete for refutation

If a propositional CNF knowledge base is unsatisfiable, resolution can derive the empty clause.
Given enough search, a contradiction can be found.

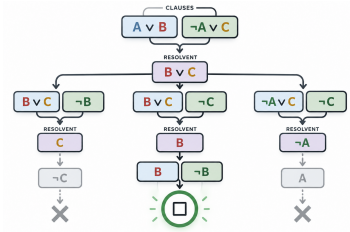
But complete does not mean cheap

The number of possible clauses and resolution steps can still grow explosively.

Resolution Is Also a Search Problem

At each step, an inference engine must decide:

- ▶ Which two clauses should be resolved?
- ▶ Which complementary literal should be used?
- ▶ Which derived clauses should be kept?
- ▶ When should the search stop?



Key idea

Logical inference does not escape search. It changes the search space from world states to proofs and symbolic consequences.

A Useful Restriction: Horn Clauses

General CNF is expressive, but unrestricted resolution can be expensive.

A **Horn clause** contains at most one positive literal.

Examples:

$$\neg A \vee \neg B \vee C$$

$$\neg A \vee D$$

$$E$$

These have a convenient rule interpretation.

Horn Clauses Look Like Rules

Consider

$$\neg A \vee \neg B \vee C.$$

This is equivalent to

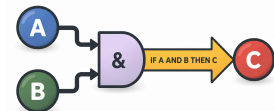
$$(A \wedge B) \rightarrow C.$$

We can read it as

IF A and B , THEN C .

Rule-system vocabulary

A and B are **premises**. C is the **conclusion** or **head**.



Why Horn Clauses Matter in AI

Horn rules support a simple and efficient style of inference.

- ▶ Facts can be stored directly.
- ▶ Rules have a natural IF–THEN interpretation.
- ▶ New facts can be generated by matching satisfied premises.
- ▶ The same rule representation supports both forward and backward reasoning.

Historical connection

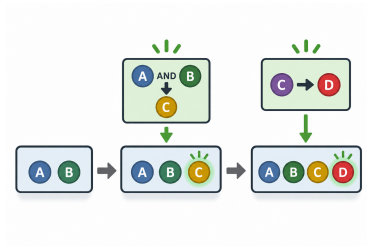
Rule-based expert systems and logic-programming languages made this style of symbolic reasoning a central part of classical AI.

Forward Chaining: Start with What You Know

Definition

Forward chaining repeatedly applies rules whose premises are known to be true, adding their conclusions to the knowledge base.

Facts → applicable rules → new facts → more rules



Key idea

Forward chaining is data-driven: begin with available evidence and ask what follows from it.

Forward Chaining Example: A Maintenance Agent

Initial facts:

BatteryLow, ChargerConnected, Docked

Rules:

BatteryLow $\wedge$ ChargerConnected $\rightarrow$ Charging

Charging $\wedge$ Docked $\rightarrow$ SafeToPowerDown

SafeToPowerDown $\rightarrow$ MaintenanceMode

Question

What new facts will forward chaining derive?

Forward Chaining Trace

1. *BatteryLow* and *ChargerConnected* are known.

$\Rightarrow$ *Charging*

2. *Charging* and *Docked* are now known.

$\Rightarrow$ *SafeToPowerDown*

3. *SafeToPowerDown* is known.

$\Rightarrow$ *MaintenanceMode*

Result

The agent derives *MaintenanceMode* even though that fact was never explicitly entered into the original knowledge base.

An Efficient Forward-Chaining Implementation

A practical implementation avoids rescanning every rule from scratch.

- ▶ Maintain an **agenda** of newly discovered facts.
- ▶ For each rule, track how many premises are still unsatisfied.
- ▶ When the count reaches zero, add the rule's conclusion to the agenda.
- ▶ Never process the same inferred fact twice.

Key idea

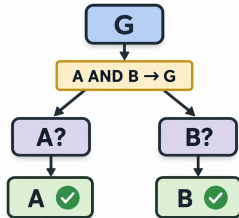
For propositional Horn knowledge bases, forward chaining can be implemented in time linear in the size of the knowledge base.

Backward Chaining: Start with What You Want to Prove

Definition

Backward chaining begins with a query and recursively asks which rules could establish that query.

Goal $\rightarrow$ rules that conclude the goal $\rightarrow$ required subgoals



Key idea

Backward chaining is goal-driven: explore only the parts of the knowledge base relevant to the current query.

Backward Chaining Example

Query:

MaintenanceMode?

The only rule that can produce it is

SafeToPowerDown $\rightarrow$ *MaintenanceMode*.

So backward chaining creates the subgoal

SafeToPowerDown?

To prove that, it finds

Charging $\wedge$ *Docked* $\rightarrow$ *SafeToPowerDown*,

producing two more subgoals:

Charging? *Docked?*

Backward Chaining Continues Until It Hits Facts

We already know

Docked.

To prove *Charging*, use

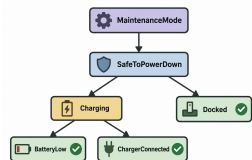
$BatteryLow \wedge ChargerConnected \rightarrow Charging.$

Both premises are known facts:

BatteryLow, *ChargerConnected*.

Therefore the proof succeeds all the way back up:

MaintenanceMode.



Forward vs. Backward Chaining

	Forward chaining	Backward chaining
Starts from	known facts	a query / goal
Direction	data $\rightarrow$ consequences	goal $\rightarrow$ prerequisites
Good when	many consequences may matter	one specific query matters
Risk	derive many irrelevant facts	revisit subgoals / chase unproductive rules

Key idea

The knowledge representation may be the same; the search strategy through the rule space is different.

Logical Agents Can Mix Inference Strategies

A practical logical agent does not need one universal inference procedure.

- ▶ Use forward chaining to maintain obvious consequences of new sensor data.
- ▶ Use backward chaining when answering a specific query.
- ▶ Use resolution when general CNF reasoning is required.
- ▶ Use specialized algorithms when the knowledge representation permits them.

AI engineering lesson

Representation and inference are tightly coupled. A restricted representation can make reasoning dramatically cheaper.

The GOFAI Connection: Intelligence as Proof and Rules

Much of classical symbolic AI was built around the idea that intelligent behavior could emerge from:

explicit knowledge + symbolic rules + automated inference

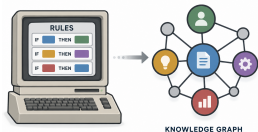
This produced theorem provers, expert systems, planning systems, and logic-programming environments.

Strength

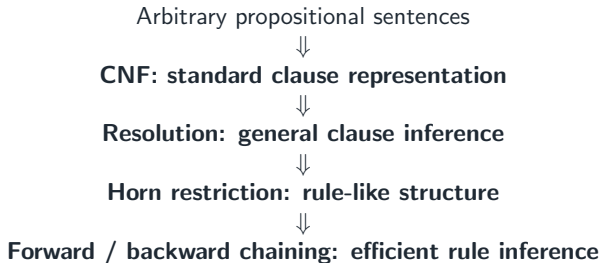
Reasoning steps can be explicit, inspectable, and justified.

Limitation

Real-world knowledge can be difficult to encode completely and brittle when assumptions change.



Putting the Lecture Together



Key idea

Automated inference is the search for consequences. The structure of the knowledge base determines which inference algorithms are available and how expensive that search becomes.

Where Propositional Logic Hits a Wall

Suppose we want to represent:

Every student enrolled in AI has access to the AI lab.

In propositional logic, we would need separate statements for every student:

EnrolledAlice $\rightarrow$ *AccessAlice*

EnrolledBob $\rightarrow$ *AccessBob*

EnrolledCarol $\rightarrow$ *AccessCarol*

$\vdots$

Next lecture

First-order logic introduces objects, predicates, relations, and quantifiers so one rule can describe an entire category of entities.

Takeaways and Next Time

- ▶ CNF rewrites propositional sentences into a conjunction of clauses.
- ▶ Resolution combines clauses by eliminating complementary literals.
- ▶ Resolution refutation proves $KB \models Q$ by deriving a contradiction from $KB \wedge \neg Q$.
- ▶ Horn clauses provide a restricted rule-like form that supports efficient inference.
- ▶ Forward chaining is data-driven; backward chaining is goal-driven.
- ▶ Logical inference is still a search problem—but now the search is over proofs, rules, and consequences.
- ▶ Next: **First-Order Logic and Knowledge Representation.**

Key idea

The central tradeoff is expressive power versus inference cost: richer representations can say more, but often require more sophisticated reasoning.